Chapter 7
Synchronization Examples

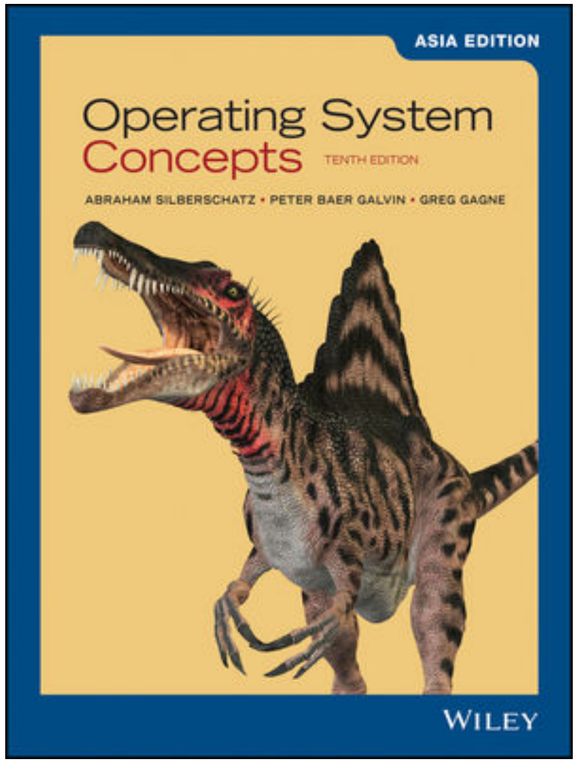
Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/5/13

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts, Asia Edition,” 10/e, ISBN: 978-1-119-58616-6, Wiley



Outline

- Classic Problems of Synchronization
- Synchronization in Java

Outline

→重要：哲學家晚餐問題

- Classic Problems of Synchronization
- Synchronization in Java

Class Problems of Synchronization

- Problems
  - The Bounded-Buffer Problem (like：Ch3、Produer - Consumer Problem)
  - The Readers-Writers Problem
  - The Dining-Philosophers Problem

- These problems are used for testing nearly every newly proposed synchronization scheme

Class Problems of Synchronization

- Problems
  - The Bounded-Buffer Problem
  - The Readers-Writers Problem
  - The Dining-Philosophers Problem

The Bounded-Buffer Problem

相比 CH3 更為複雜，考慮了不只一个 Producer和Consumer的情況

- The producer and consumer processes share the following data structures
  - semaphore mutex = 1; (Binary) (Producer、Consumer 共用)
  - semaphore empty = BUFFER_SIZE; (Counting)
  - semaphore full = 0; (Counting)

(或 3 个 Spin Lock)

The Bounded-Buffer Problem

Producer

```
while (true) {
      . . .
   /* produce an item in next_produced */
      . . .
   wait(empty);
   wait(mutex);
      . . .
   /* add next_produced to the buffer */
      . . .
   signal(mutex);
   signal(full);
}
```

Consumer

```
while (true) {
   wait(full);
   wait(mutex);
      . . .
   /* remove an item from buffer to next_consumed */
      . . .
   signal(mutex);
   signal(empty);
      . . .
   /* consume the item in next_consumed */
      . . .
}
```

semaphore mutex = 1;

semaphore empty = BUFFER_SIZE;

semaphore full = 0;

```
wait(S) {                          signal(S) {
    while (S <= 0)                     S++;
       ; // busy wait                }
    S--;
}
```

Thinking

- What happened when we have multiple producers/consumers (w/o mutex)
  - Two producers decrement empty
  - One of the producers determines the next empty slot in the buffer
  - Second producer determines the next empty slot and gets the same result as the first producer
  - Both producers write into the same slot
  - Source: https://en.wikipedia.org/wiki/Producer%E2%80%93consumer_problem

Class Problems of Synchronization

- Problems
  - The Bounded-Buffer Problem
  - The Readers-Writers Problem
  - The Dining-Philosophers Problem

The Readers-Writers Problem

Often happened in site / database

- Database
  - Reader (Read Only)
  - Writer (Read and Write)

- If two readers access the shared data simultaneously, no adverse effects will result.
- However, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue.
- We require that the writers have exclusive access to the shared database while writing to the database

Variations

- Priorities
- The *first* readers-writers problem
  - No reader be kept waiting unless a writer has already obtained permission to use the shared object
  - Writers may starve
- The *second* readers-writers problem
  - Once a writer is ready, the writer performs its write as soon as possible
  - Readers may starve
- Referene: https://en.wikipedia.org/wiki/Readers%E2%80%93writers_problem

The *first* readers-writers problem

- Shared Data Structures
  - semaphore rw_mutex = 1; (Binary)
    - common to both reader and writer processes
  - semaphore mutex = 1; (Binary)
    - ensure mutual exclusion when the variable read_count is updated
  - int read_count = 0; (Counting)

The *first* readers-writers problem

曾經考过

Disadvantage：很容易讓writer永遠輸不到
(只要有一个reader还在读，writer就永遠不能寫)

Writer

```
while (true) {
   wait(rw_mutex);
      . . .
   /* writing is performed */
      . . .
   signal(rw_mutex);
}
```

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

```
signal(S) {
    S++;
}
```

Reader

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

現在沒有reader在讀資料

→ 不是 C、S、可以有多个 Process 同時進來

The *first* readers-writers problem

Reader 1

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

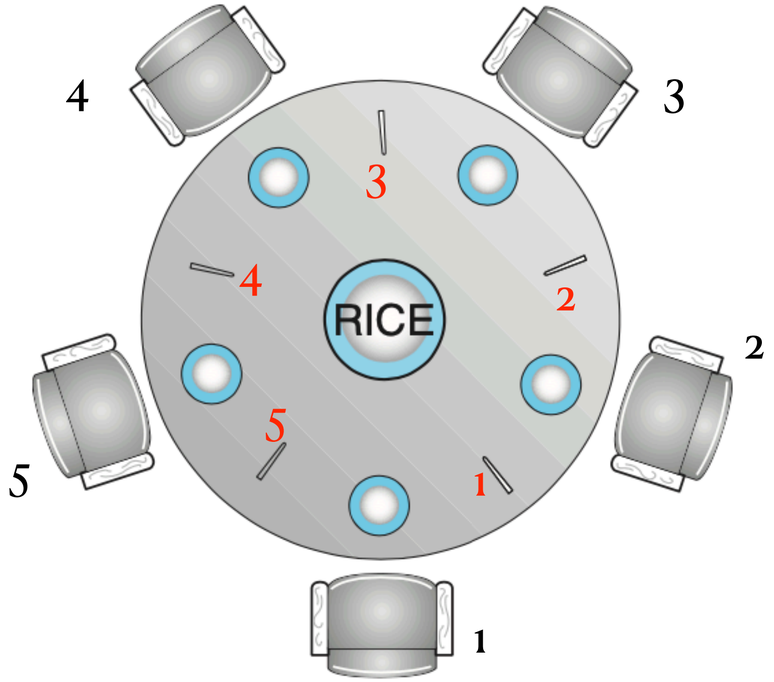
Reader 2

```
while (true) {
   wait(mutex);
   read_count++;
   if (read_count == 1)
      wait(rw_mutex);
   signal(mutex);
      . . .
   /* reading is performed */
      . . .
   wait(mutex);
   read_count--;
   if (read_count == 0)
      signal(rw_mutex);
   signal(mutex);
}
```

Class Problems of Synchronization

- Problems
  - The Bounded-Buffer Problem
  - The Readers-Writers Problem
  - The Dining-Philosophers Problem 哲學家晚餐問題

The Dining-Philosophers Problem



*哲學家 → Process
筷子 → Spin Lock / Semaphore
飯 → Critical Section

Deadlock
Starvation

Semaphore Solution

- One simple solution is to represent each chopstick with a semaphore
  - semaphore chopstick[5];

```
while (true) {
   wait(chopstick[i]);
   wait(chopstick[(i+1) % 5]);
      . . .
   /* eat for a while */
      . . .
   signal(chopstick[i]);
   signal(chopstick[(i+1) % 5]);
      . . .
   /* think for awhile */
      . . .
}
```

Semaphore Solution

- This solution guarantees that no two neighbors are eating simultaneously
- It could create a deadlock
- Several possible remedies
  - Allow at most four philosophers to be sitting simultaneously at the table
  - Allow a philosopher to pick up her chopsticks only if both chopsticks are available (to do this, she must pick them up in a critical section)
  - Use an asymmetric solution
    - An odd-numbered philosopher picks up first her left chopstick and then her right chopstick
    - An even-numbered philosopher picks up her right chopstick and then her left chopstick
- A deadlock-free solution does not necessarily eliminate the possibility of starvation

Outline

- Classic Problems of Synchronization
- Synchronization in Java
  - Java Monitors (synchronized, wait, notify)
  - Reentrant Locks
  - Semaphores

Example

```
public synchronized static int storeRecord(Context context,
        String color, int shift, int notshow, int starttime,
        int order) {

    DBOpenHelper dbHelper = new DBOpenHelper(context);
    SQLiteDatabase db = dbHelper.getWritableDatabase();

    ContentValues cv = new ContentValues();
    cv.put(SHIFTNAME, name);
    cv.put(COLOR, color);
    cv.put(SHIFT, shift);
    cv.put(NOTSHOW, notshow);
    cv.put(STARTTIME, starttime);
    cv.put(ENDTIME, endtime);
    cv.put(ORDER, order);

    if (shift == -1) { // New Record
        shift = searchShiftNumber(db);
        cv.put(SHIFT, shift);
        db.insert(SHIFT_TABLE, nullColumnHack: null, cv);
```

synchronized

- A monitor-like concurrency mechanism
- Every **object** in Java has associated with it a single lock
- When a method is declared to be synchronized, calling the method requires owning the lock for the **object**
  - Singleton (https://en.wikipedia.org/wiki/Singleton_pattern)

A simple example (SyncExample.java)

```
public class SyncExample {

    int x;

    public static void main(String[] args) {
        SyncExample example = new SyncExample();
        example.main();
    }

    void main() {

        this.x = 3;

        AddThread t1 = new AddThread(this, 1);
        AddThread t2 = new AddThread(this, 2);

        t1.start();
        t2.start();
        try {
            Thread.sleep(6000);
        } catch (InterruptedException e) {
        }
        System.out.println(x);
    }
```

In this example, we have 3 objects and 3 threads.

```
class AddThread extends Thread {

    int addValue = 0;
    SyncExample example;

    public AddThread(SyncExample example, int addValue) {
        this.addValue = addValue;
        this.example = example;
    }

    @Override
    public void run() {
        example.addX(addValue);
    }
}
```

The addX method should be SyncExample's method, and cannot be AddThread's method

```
synchronized void addX(int addValue) {
    int localX = x;
    localX += addValue;
    long milliseconds = (long)(Math.random()*3000);
    try {
        Thread.sleep(milliseconds);
    } catch (InterruptedException e) {
    }
    x = localX;
}
```

A simple example (SyncExample.java)

```
addValue: 2, milliseconds: 807
addValue: 1, milliseconds: 329
5

addValue: 2, milliseconds: 285
addValue: 1, milliseconds: 318
4
```

```
void addX(int addValue) {
// synchronized void addX(int addValue) {
    int localX = x;
    localX += addValue;
    long milliseconds = (long)(Math.random()*3000);
    System.out.println("addValue: " + addValue +
            ", milliseconds: " + milliseconds);
    try {
        Thread.sleep(milliseconds);
    } catch (InterruptedException e) {
    }
    x = localX;
}
```

synchronized static

SyncStaticExample.java
OutsideAddThread.java

```
public class OutsideAddThread extends Thread {

    @Override
    public void run() {
        addX(addValue);
    }

    // synchronized static void addX(int addValue) {
    synchronized void addX(int addValue) {
        int localX = example.x;
        localX += addValue;
        long milliseconds = (long)(Math.random()*3000);
        System.out.println("addValue: " + addValue +
            ", milliseconds: " + milliseconds);
        try {
            Thread.sleep(milliseconds);
        } catch (InterruptedException e) {
        }
        example.x = localX;
    }
```

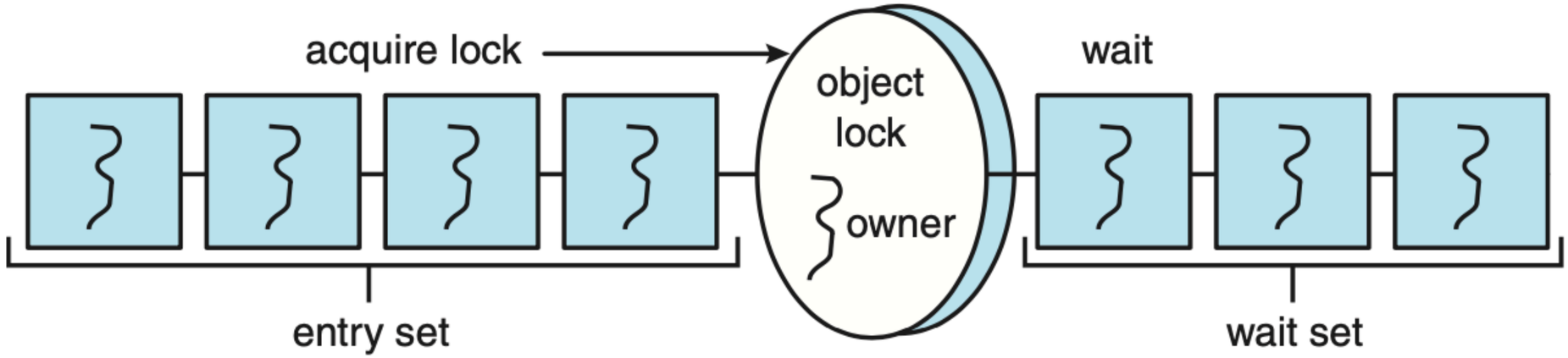
wait / notify

- When a thread calls the wait() method, the following happens:
  - The thread releases the lock for the object
  - The state of the thread is set to blocked
  - The thread is placed in the wait set for the object

- The call to notify():
  - Picks an arbitrary thread T from the list of threads in the wait set
  - Moves T from the wait set to the entry set
  - Sets the state of T from blocked to runnable

- https://www.baeldung.com/java-wait-notify

wait / notify



BoundedBuffer.java (1/2)

```
public class BoundedBuffer {

    private static final int BUFFER_SIZE = 3;

    private int count = 0, in = 0, out = 0;
    private String [] buffer = {"", "", ""};

    public synchronized void insert(String item) {
        while (count == BUFFER_SIZE) {
            try {
                System.out.println("Insert Wait");
                wait();
            } catch (InterruptedException e) {
            }
        }

        System.out.println("Insert: " + item + ", " + in);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;
    }
```

BoundedBuffer.java (2/2)

```
    public synchronized String remove() {
        String item;

        while (count == 0) {
            try {
                wait();
            } catch (InterruptedException e) {
            }
        }

        item = buffer[out];

        System.out.println("Remove: " + item + ", " + out);

        out = (out + 1) % BUFFER_SIZE;
        count--;

        notify();

        return item;
    }
}
```

BBMain.java

```
public class BBMain {

    BoundedBuffer buffer;

    public static void main(String[] args) {
        BBMain mainthread = new BBMain();
        mainthread.buffer = new BoundedBuffer();
        mainthread.main();
    }

    void main() {
        ProducerThread p1 = new ProducerThread("A");
        ProducerThread p2 = new ProducerThread("B");
        ProducerThread p3 = new ProducerThread("C");
        ProducerThread p4 = new ProducerThread("D");

        ConsumerThread c1 = new ConsumerThread();
        ConsumerThread c2 = new ConsumerThread();
        ConsumerThread c3 = new ConsumerThread();
        ConsumerThread c4 = new ConsumerThread();

        p1.start();
        p2.start();
        p3.start();
```

```
class ProducerThread extends Thread {

    String insertItem;

    ProducerThread(String item) {
        this.insertItem = item;
    }

    @Override
    public void run() {
        buffer.insert(insertItem);
    }
}

class ConsumerThread extends Thread {
    @Override
    public void run() {
        String item = buffer.remove();
    }
}
```

BBMain.java

```
Insert: A, 0
Insert: B, 1
Insert: C, 2
Insert Wait
Remove: A, 0
Insert: D, 0
Remove: B, 1
Remove: C, 2
Remove: D, 0
```

```
Insert: A, 0
Insert: D, 1
Remove: A, 0
Insert: C, 2
Insert: B, 0
Remove: D, 1
Remove: C, 2
Remove: B, 0
```

ReentrantLock

```
Lock key = new ReentrantLock();

key.lock();
try {
   /* critical section */
}
finally {
   key.unlock();
}
```

ReentrantLock

```
void addX(int addValue) {
    key.lock();
    int localX = x;
    localX += addValue;
    long milliseconds = (long)(Math.random()*3000);
    System.out.println("addValue: " + addValue +
            ", milliseconds: " + milliseconds);
    try {
        Thread.sleep(milliseconds);
    } catch (InterruptedException e) {
    }
    x = localX;
    key.unlock();
}
```

Semaphore

```
Semaphore sem = new Semaphore(1);

try {
   sem.acquire();
   /* critical section */
}
catch (InterruptedException ie) { }
finally {
   sem.release();
}
```

Semaphore

`public Semaphore(int permits)`

Creates a `Semaphore` with the given number of permits and nonfair fairness setting.

Parameters:

`permits` - the initial number of permits available. This value may be negative, in which case releases must occur before any acquires will be granted.

Reference: https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/concurrent/Semaphore.html

Semaphore

```
Semaphore mutex = new Semaphore(1);

void addX(int addValue) {
    try {
        mutex.acquire();
    } catch (InterruptedException e1) {
    }
    int localX = x;
    localX += addValue;
    long milliseconds = (long)(Math.random()*3000);
    System.out.println("addValue: " + addValue +
            ", milliseconds: " + milliseconds);
    try {
        Thread.sleep(milliseconds);
    } catch (InterruptedException e) {
    }
    x = localX;
    mutex.release();
}
```

Homework

- Rewrite the BoundedBuffer.java by using Semaphores
  - Hint:

```
while (true) {
        . . .
   /* produce an item in next_produced */
        . . .
   wait(empty);
   wait(mutex);
        . . .
   /* add next_produced to the buffer */
        . . .
   signal(mutex);
   signal(full);
}
```

```
while (true) {
   wait(full);
   wait(mutex);
        . . .
   /* remove an item from buffer to next_consumed */
        . . .
   signal(mutex);
   signal(empty);
        . . .
   /* consume the item in next_consumed */
        . . .
}
```

Homework

- Download Java codes
- Run the codes
- Write a test report